Concordia University · Gina Cody School of Engineering

Department of Mechanical, Industrial and Aerospace Engineering

INDU 211: Introduction to Production and Manufacturing Systems

Master Quantitative Engineering Guide: Factorial Complexity, Traveling Salesperson Problem (TSP) & Combinatorial Routing Heuristics

> **IMPORTANT**
> **Curriculum Alignment**: This master quantitative decision guide directly integrates the official Concordia University INDU 211 curriculum (Chapter 5 — Material Handling, Distribution & Routing), the professor's lecture slides (`5.0.INDU_211_CH5_2025.pdf`), and the official examination archives (Fall 2020 Midterm Problem 2 and Fall 2025 Midterm Paper A Problem 4). All derivations, heuristic tracing protocols, and optimality diagnostics are presented with complete mathematical rigor.

Table of Contents

1. Intuitive Conceptualization: What is Factorial Complexity?
2. The Factorial Explosion Law & The Curse of Dimensionality
3. Spatial Geometry & Asymmetric Matrix Navigation Rules
4. Teacher Lecture Notes Extraction & Method Analysis (Chapter 5)
5. 4.1 Mathematical Formulation of the TSP
6. 4.2 The Nearest Neighbor (NN) Heuristic Algorithm
7. 4.3 Teacher Lecture Example 1 (Stops A through G) & The Greedy Trap
8. 4.4 Clark-Wright Savings Heuristic (Vehicle Routing Problem)
9. Official Past Exam 1: Fall 2020 Midterm (Warehouse Forklift Routing)
10. Problem Statement & Asymmetric Matrix
11. Question 2.1: Nearest-Neighbor Tour S1 (Detailed Trace)
12. Question 2.2: Second-Nearest First Step Tour S2
13. Question 2.3: Optimality Verdict & Proof
14. Question 2.4: Combinatorial Factorial Enumeration (5 vs. 20 Departments)
15. Official Past Exam 2: Fall 2025 Midterm Paper A (AGV Machine Routing)
16. Problem Statement & Visual AGV Layout
17. Question 4.1: Sequence S1 from Storage S ($T_1 = 9.0$ min)

18. Question 4.2: Sequence S2 from Machine M1 ($T_2 = 7.9$ min)
19. Question 4.3: Diagnostic True / False / MTF Analysis
20. Question 4.4: The Multi-Start Nearest Neighbor Controversy
21. Exhaustive Algorithmic Benchmark (All 120 Tours Proven)
22. The Industrial Engineer's Exam Diagnostic & Failure-Mode Prevention

---

1. Intuitive Conceptualization: What is Factorial Complexity?

In material handling and supply chain engineering, goods, forklifts, and automated guided vehicles (AGVs) must constantly visit a sequence of workstations, inventory racks, or delivery destinations before returning to their home base.

The Decision Tree Analogy

Imagine an automated guided vehicle (AGV) stationed at a central depot P. It has to deliver parts to k different machines: 1. For its **first stop**, the vehicle can choose any of the k unvisited machines (k choices). 2. Once at the first machine, it cannot re-visit it. It now has $(k-1)$ remaining machines to choose from. 3. From the second machine, it has $(k-2)$ remaining choices. 4. This continues down to the last machine, which has only 1 remaining choice, after which it must travel back to depot P.

By the fundamental counting principle of combinatorics, the total number of distinct routing paths that visit every machine is the product of the number of choices at each branch:

$$\text{Total Directed Paths} = k \times (k-1) \times (k-2) \times \cdots \times 2 \times 1 = k!$$

If the network has n total locations (1 depot plus $n-1$ customer or machine stations), fixing the starting point leaves $n-1$ stations to sequence:

$$\text{Total Closed Directed Tours} = (\mathbf{n-1})!$$

Why "Factorial" is Different from "Polynomial" or "Exponential"

- **Polynomial Growth (n^2, n^3)**: Doubling the problem size increases runtime by a constant factor ($2^2 = 4\times$).
- **Exponential Growth (2^n)**: Each added station doubles the search space.
- **Factorial Growth ($n!$)**: Each added station multiplies the entire existing search space by n. Factorials grow faster than any simple exponential base!

---

2. The Factorial Explosion Law & The Curse of Dimensionality

To understand why industrial engineers cannot simply "use a computer to test all possibilities" on real production lines, examine how $(n-1)!$ escalates:

Total Nodes (n)	Unvisited Stations ($n-1$)	Directed Permutations $(n-1)!$	Symmetric Tours $\frac{(n-1)!}{2}$	Time at 1 Billion Tours / Second
4	3	$3! = \mathbf{6}$	3	0.000000006 seconds
5	4	$4! = \mathbf{24}$	12	0.000000024 seconds
6 *(Fall 2025 Midterm)*	5	$5! = \mathbf{120}$	60	0.000000120 seconds
7 *(Teacher Example 1)*	6	$6! = \mathbf{720}$	360	0.000000720 seconds
11	10	$10! = \mathbf{3,628,800}$	1,814,400	0.0036 seconds
16	15	$15! \approx \mathbf{1.31 \times 10^{12}}$	6.54×10^{11}	21.8 minutes
21 *(Fall 2020 Exam Q2.4)*	20	$20! \approx \mathbf{2.43 \times 10^{18}}$	1.22×10^{18}	**77.1 YEARS**
31	30	$30! \approx \mathbf{2.65 \times 10^{32}}$	1.33×10^{32}	8.4×10^{15} **YEARS**

> **CAUTION**
> **The Engineering Takeaway**: At $n = 21$ locations (a modest city courier route or a standard manufacturing shop floor), testing every path by brute-force evaluation would take over **77 years** on a supercomputer evaluating one billion routes per second. At $n = 31$, it exceeds the age of the universe! This mathematical reality is called the **Curse of Dimensionality** and proves why the Traveling Salesperson Problem (TSP) is **NP-hard**. Industrial Engineers must therefore rely on **heuristic algorithms** (e.g., Nearest Neighbor, Clark-Wright Savings, 2-Opt).

---

3. Spatial Geometry & Asymmetric Matrix Navigation Rules

Why Real Facilities Have Asymmetric Matrices ($c_{ij} \neq c_{ji}$)

In pure Euclidean mathematics, the distance from point A to point B is identical to the distance from point B to point A ($d_{AB} = d_{BA}$). However, industrial factory layouts and distribution warehouses are characterized by **directional asymmetry**: 1. **One-Way Aisles**: Forklift safety regulations enforce strict unidirectional traffic lanes to prevent head-on collisions. 2. **Material Handling Equipment**: Overhead cranes, incline roller conveyors, and gravity chutes operate in a single direction. 3. **Elevator and Vertical Transfers**: Moving up floors in multi-story facilities incurs different delays and travel paths than moving down.

When $c_{ij} \neq c_{ji}$, the distance matrix is **asymmetric**.

The Golden Rule of Matrix Reading

Every cell in an operational distance or travel-time matrix is indexed as:

$$c_{ij} = \text{Cost/Time/Distance from Node } i \text{ (Row) to Node } j \text{ (Column)}$$

```
          Destination (To Node j)  ⟶ [Read Columns]
                 P      A      B      C      D      E
Origin
(From Node i)  ┌──────┬──────┬──────┬──────┬──────┬──────┐
   ▲           │  -   │  15  │  18  │  20  │  25  │  13  │ ⟵ Row P
   │           ├──────┼──────┼──────┼──────┼──────┼──────┤
[Read Rows]    │  24  │  -   │  12  │  16  │  13  │  14  │ ⟵ Row A
               └──────┴──────┴──────┴──────┴──────┴──────┘
```

> **WARNING**
>
> **CRITICAL EXAM TRAP**: * **Always read HORIZONTALLY along the ROW** of the node where the vehicle currently resides. * **Never read VERTICALLY down the column**! Reading down the column gives travel *into* the current station from other stations, which produces completely incorrect tours and costs.

---

4. Teacher Lecture Notes Extraction & Method Analysis (Chapter 5)

4.1 Mathematical Formulation of the TSP

From teacher lecture deck `5.0.INDU_211_CH5_2025.pdf` (Slide 13): * Let $V = 1, 2, \ldots, n$ be the set of physical nodes (depot and customer/machine stations). * Let c_{ij} be the cost, distance, or travel time from station i to station j. * Let decision variable $x_{ij} \in 0, 1$, where $x_{ij} = 1$ if the route goes directly from i to j, and 0 otherwise.

$$\min Z = \sum_{i=1}^{n} \sum_{j=1}^{n} c_{ij} x_{ij}$$

Subject to: 1. **Outflow Constraint**: Exactly one departure from every station:

$$\sum_{j=1, j \neq i}^{n} x_{ij} = 1 \quad \forall\, i \in V$$

2. **Inflow Constraint**: Exactly one arrival at every station:

$$\sum_{i=1, i \neq j}^{n} x_{ij} = 1 \quad \forall\, j \in V$$

3. **Subtour Elimination**: Ensures the solution forms a single closed loop of n stations rather than disjoint disconnected sub-loops (e.g., two independent 3-node triangles).

4.2 The Nearest Neighbor (NN) Heuristic Algorithm

The Nearest Neighbor heuristic is a greedy construction heuristic that builds a tour step-by-step: 1. **Initialization**: Select an initial start node v_0 (e.g., the depot P). Set current node $u = v_0$. Mark v_0 as visited. Initialize $\text{Tour} = [v_0]$ and $\text{Cost} = 0$. 2. **Greedy Step**: Among all unvisited nodes $v \notin \text{Visited}$, find node $v^ that minimizes travel cost c_{uv}: $v^ = \arg\min_{v \notin \text{Visited}} c_{uv}$$ 3. **Transition**: Add $c_{u v^} to total cost. Move to v^: set u = v^, mark v^ as visited, and append v^ to \text{Tour}$. 4. *Termination Check: If unvisited nodes remain, repeat Steps 2 and 3. 5. Closure*: Once all nodes are visited, add the return leg back to the origin:

$$\text{Cost} = \text{Cost} + c_{uv_0}, \quad \text{Tour} = \text{Tour} \cup [v_0]$$

4.3 Teacher Lecture Example 1 (Stops A through G) & The Greedy Trap

In teacher lecture slide 14, the professor analyzes a 7-station distribution network with depot A and stops B, C, D, E, F, G:

Symmetric Distance Table:

$$\begin{array}{c|ccccccc} \text{Node} & A & B & C & D & E & F & G \\ \hline A & — & 14 & 21 & 20 & 6 & 24 & 9 \\ B & 14 & — & 10 & 9 & 9 & 10 & 11 \\ C & 21 & 10 & — & 1 & 15 & 9 & 21 \\ D & 20 & 9 & 1 & — & 14 & 9 & 20 \\ E & 6 & 9 & 15 & 14 & — & 19 & 9 \\ F & 24 & 10 & 9 & 9 & 19 & — & 21 \\ G & 9 & 11 & 21 & 20 & 9 & 21 & — \end{array}$$

Trace of the Nearest Neighbor Heuristic from Depot A:

1. From A: Unvisited B, C, D, E, F, G. Minimum is to **E** ($c_{AE} = 6$).
2. From E: Unvisited B, C, D, F, G. Distances: $B = 9, C = 15, D = 14, F = 19, G = 9$.
3. **Tie Condition**: Both B and G have distance 9.
4. If we break the tie to **B** ($c_{EB} = 9$):
5. From B: Unvisited C, D, F, G. Minimum is to **D** ($c_{BD} = 9$).
6. From D: Unvisited C, F, G. Minimum is to **C** ($c_{DC} = 1$).
7. From C: Unvisited F, G. Minimum is to **F** ($c_{CF} = 9$).
8. From F: Only **G** remains. Forced choice: $c_{FG} = 21$!
9. Return to A: From $G \to A$ ($c_{GA} = 9$).

$$\text{NN Tour} : A \to E \to B \to D \to C \to F \to G \to A$$

$$\text{Total Distance} = 6 + 9 + 9 + 1 + 9 + 21 + 9 = \mathbf{64}$$

Comparison with Global Optimum:

The true global minimum tour is:

$$\text{Optimal Tour} : A \to G \to B \to F \to C \to D \to E \to A$$

$$\text{Optimal Distance} = 9 + 11 + 10 + 9 + 1 + 14 + 6 = \mathbf{60}$$

> **NOTE**
> **The Greedy Trap Explained**: Nearest Neighbor achieved 64, which is **6.7% worse** than optimal. Why? Because it greedily took short legs early ($A \to E = 6, D \to C = 1$), leaving station G isolated at the very end. The vehicle was forced to make a massive, expensive traverse from $F \to G$ (21 units). Breaking the earlier tie at E toward G yields an even worse tour of **69**. This confirms that greedy local heuristics are myopic and cannot look ahead.

4.4 Clark-Wright Savings Heuristic (Vehicle Routing Problem)

When multiple delivery vehicles with carrying capacities operate from a central depot 0, the professor presents the **Clark-Wright Savings Heuristic** (Slides 16–21):

The Savings Concept:

If two customer stops i and j are served on separate out-and-back trips from depot 0:

$$\text{Separate Distance} = (c_{0i} + c_{i0}) + (c_{0j} + c_{j0}) = 2c_{0i} + 2c_{0j}$$

If customer stops i and j are consolidated onto a single combined route ($0 \to i \to j \to 0$):

$$\text{Combined Distance} = c_{0i} + c_{ij} + c_{j0}$$

The net travel distance saved by joining stops i and j into one route is:

$$\mathbf{S_{ij} = c_{0i} + c_{0j} - c_{ij}}$$

Algorithmic Execution:

1. Calculate the savings S_{ij} for all distinct pairs (i, j).
2. Sort all savings values in **descending numerical order**.
3. Starting from the largest savings, link stations i and j if and only if:
4. Neither station is already interior to an existing route (both must be adjacent to the depot / at route endpoints).
5. Stations i and j are not already on the same route.
6. The combined total delivery demand does not exceed the vehicle's capacity:

$$\sum_{k \in \text{Route}} q_k \leq \text{Capacity}$$

7. Repeat until all stops are serviced.

---

5. Official Past Exam 1: Fall 2020 Midterm (Warehouse Forklift Routing)

5.1 Problem Statement & Asymmetric Matrix

(Midterm Exam Fall 2020, Problem 2 — 18 Marks): A warehouse forklift starts at central depot **P** and must visit 5 production departments (**A, B, C, D, E**) exactly once to replenish raw inventory totes before returning to **P**. Because of strict one-way corridor routing, travel distances are asymmetric:

Distance Matrix (d_{ij} in meters):

$$\begin{array}{c|ccccc} \text{From \ To} & P & A & B & C & D & E \\ \hline P & — & 15 & 20 & 19 & 26 & 13 \\ A & 9 & — & 12 & 16 & 13 & 18 \\ B & 19 & 13 & — & 17 & 15 & 18 \\ C & 22 & 16 & 12 & — & 16 & 14 \\ D & 18 & 10 & 18 & 20 & — & 12 \\ E & 22 & 9 & 11 & 13 & 14 & — \end{array}$$

5.2 Question 2.1: Nearest-Neighbor Tour S1 (Detailed Trace)

Problem: Starting at depot P, use the Nearest Neighbor method to construct route S_1 and determine total distance T_1.

- **Step 1 (At P)**:
- Candidates: A, B, C, D, E.
- Read Row P: $d_{PA} = 15, d_{PB} = 20, d_{PC} = 19, d_{PD} = 26, d_{PE} = \mathbf{13}$.
- Minimum is **E** ($d_{PE} = 13$). Visited: P, E.
- **Step 2 (At E)**:
- Candidates: A, B, C, D.
- Read Row E: $d_{EA} = \mathbf{9}, d_{EB} = 11, d_{EC} = 13, d_{ED} = 14$.
- Minimum is **A** ($d_{EA} = 9$). Visited: P, E, A.
- **Step 3 (At A)**:
- Candidates: B, C, D.
- Read Row A: $d_{AB} = \mathbf{12}, d_{AC} = 16, d_{AD} = 13$.
- Minimum is **B** ($d_{AB} = 12$). Visited: P, E, A, B.
- **Step 4 (At B)**:
- Candidates: C, D.
- Read Row B: $d_{BC} = 17, d_{BD} = \mathbf{15}$.
- Minimum is **D** ($d_{BD} = 15$). Visited: P, E, A, B, D.
- **Step 5 (At D)**:
- Only **C** remains.
- Read Row D: $d_{DC} = \mathbf{20}$. Visited: All.
- **Step 6 (Return to P)**:
- From $C \to P$: Read Row C: $d_{CP} = \mathbf{22}$.

$$\textbf{Sequence S1} : P \to E \to A \to B \to D \to C \to P$$

$$\textbf{Total Distance T1} = 13 + 9 + 12 + 15 + 20 + 22 = \mathbf{91} \text{ meters}$$

5.3 Question 2.2: Second-Nearest First Step Tour S2

Problem: Same matrix. Go to the **second-nearest** department on the first move from P, then use the nearest-neighbor rule for every subsequent stop. Determine total distance T_2.

- **Step 1 (At P)**:
- Row P: $E = 13$ (1st nearest), $A = \mathbf{15}$ (2nd nearest).
- Move to **A** ($d_{PA} = 15$). Visited: P, A.
- **Step 2 (At A)**:
- Candidates: B, C, D, E.
- Row A: $d_{AB} = \mathbf{12}, d_{AC} = 16, d_{AD} = 13, d_{AE} = 18$.
- Nearest is **B** ($d_{AB} = 12$). Visited: P, A, B.
- **Step 3 (At B)**:
- Candidates: C, D, E.
- Row B: $d_{BC} = 17, d_{BD} = \mathbf{15}, d_{BE} = 18$.
- Nearest is **D** ($d_{BD} = 15$). Visited: P, A, B, D.
- **Step 4 (At D)**:
- Candidates: C, E.
- Row D: $d_{DC} = 20, d_{DE} = \mathbf{12}$.
- Nearest is **E** ($d_{DE} = 12$). Visited: P, A, B, D, E.
- **Step 5 (At E)**:
- Only **C** remains.
- Row E: $d_{EC} = \mathbf{13}$. Visited: All.
- **Step 6 (Return to P)**:
- From $C \to P$: Row C: $d_{CP} = \mathbf{22}$.

$$\textbf{Sequence S2}: P \to A \to B \to D \to E \to C \to P$$

$$\textbf{Total Distance T2} = 15 + 12 + 15 + 12 + 13 + 22 = \mathbf{89} \text{ meters}$$

5.4 Question 2.3: Optimality Verdict & Proof

Problem: Are the routes from 2.1 and 2.2 optimal? * **Answer**: **Neither is guaranteed optimal; both are heuristics.** * **Mathematical Proof**: 1. Route S_2 achieves a total distance of **89 meters**, which is strictly shorter than the standard Nearest Neighbor route S_1 (**91 meters**). This immediately proves that S_1 is **definitively NOT optimal**. 2. Because Nearest Neighbor is a greedy heuristic without global foresight, neither 91 nor 89 is guaranteed to be the true global minimum. Determining whether 89 is the global optimum requires exhaustive enumeration or exact integer programming (Branch and Bound).

5.5 Question 2.4: Combinatorial Factorial Enumeration (5 vs. 20 Departments)

Question 2.4.1: "*A total-search computer program that generates every possible route and picks the shortest will find the optimal route.*" * **Answer**: **True** (Total exhaustive enumeration is an exact optimization method; it evaluates every element of the feasible set).

Question 2.4.2: "*How many routes must the total search generate for depot P plus 5 departments?*" * **Answer**: $5! = 120$ routes. * **Proof**: Depot P is fixed as the origin and destination. The 5 intermediate departments A, B, C, D, E can be visited in any permutation:

$$\text{Permutations} = 5! = 5 \times 4 \times 3 \times 2 \times 1 = \mathbf{120}$$

Because distances are asymmetric ($d_{ij} \neq d_{ji}$), reverse routes are distinct, so the total is not divided by 2.

Question 2.4.3: "*How many routes must the total search generate for depot P plus 20 departments?*" * **Answer**: $20! \approx \mathbf{2.43 \times 10^{18}}$ routes. * **Proof**:

$$20! = 2,432,902,008,176,640,000 \approx \mathbf{2.43 \times 10^{18}}$$

This factorial result explicitly illustrates the computational barrier that necessitates heuristic algorithms in industrial manufacturing.

---

6. Official Past Exam 2: Fall 2025 Midterm Paper A (AGV Machine Routing)

6.1 Problem Statement & Visual AGV Layout

(Official Midterm Examination Fall 2025 Paper A, Problem 4 — 12 Marks): An Automated Guided Vehicle (AGV) departs from a factory storage room (S) to deliver tooling fixtures to 5 machining centers (M_1, M_2, M_3, M_4, M_5) and return to S with minimum total travel time.

6x6 Asymmetric Travel Time Matrix (c_{ij} in minutes):

$$\begin{array}{c|cccccc} \text{From \ To} & S & M_1 & M_2 & M_3 & M_4 & M_5 \\ \hline S & — & 0.9 & 1.4 & \mathbf{0.7} & 1.1 & 1.6 \\ M_1 & \mathbf{1.8} & — & 2.7 & 2.3 & 2.5 & 2.0 \\ M_2 & 2.1 & 1.7 & — & 0.6 & 1.8 & 1.3 \\ M_3 & 1.6 & 1.3 & 1.6 & — & \mathbf{1.0} & 1.9 \\ M_4 & 1.2 & 1.5 & 1.9 & 1.2 & — & \mathbf{1.4} \\ M_5 & 1.5 & \mathbf{1.1} & 1.3 & 1.4 & 1.7 & — \end{array}$$

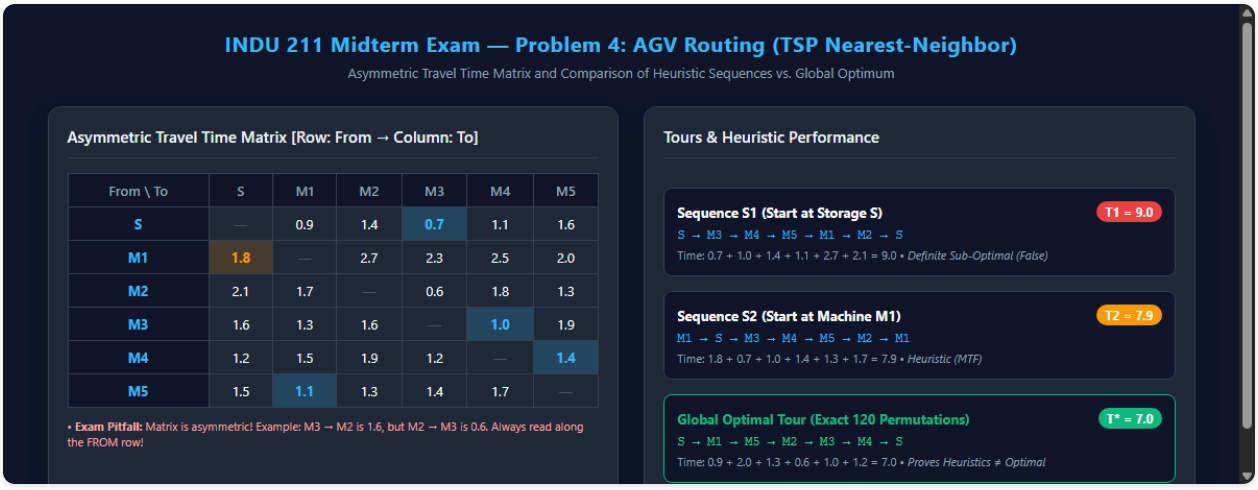


Figure 1: AGV travel time matrix and comparative routing performance for Sequence S1 ($T_1 = 9.0$), Sequence S2 ($T_2 = 7.9$), and the Global Minimum Tour ($T^=7.0$).

6.2 Question 4.1: Sequence S1 from Storage S ($T_1 = 9.0$ min)

Problem Statement:

> Assuming that the AGV is currently in the storage room (S), use the "nearest-neighbor method" as discussed in class to generate a production sequence S_1 and calculate the corresponding total travel time T_1.

Step-by-Step Traversal:

1. **At Storage Room S**:
2. Examine Row S: Destinations M_1, M_2, M_3, M_4, M_5.
3. Times: $M_1 = 0.9, M_2 = 1.4, M_3 = \mathbf{0.7}, M_4 = 1.1, M_5 = 1.6$.
4. Minimum is M_3 with $c_{S,M_3} = \mathbf{0.7}$.
5. Tour so far: $S \to M_3$. Visited: S, M_3.
6. **At Machine M3**:
7. Examine Row M_3: Unvisited destinations M_1, M_2, M_4, M_5.
8. Times: $M_1 = 1.3, M_2 = 1.6, M_4 = \mathbf{1.0}, M_5 = 1.9$.
9. Minimum is M_4 with $c_{M_3,M_4} = \mathbf{1.0}$.
10. Tour so far: $S \to M_3 \to M_4$. Visited: S, M_3, M_4.
11. **At Machine M4**:
12. Examine Row M_4: Unvisited destinations M_1, M_2, M_5.
13. Times: $M_1 = 1.5, M_2 = 1.9, M_5 = \mathbf{1.4}$.
14. Minimum is M_5 with $c_{M_4,M_5} = \mathbf{1.4}$.
15. Tour so far: $S \to M_3 \to M_4 \to M_5$. Visited: S, M_3, M_4, M_5.
16. **At Machine M5**:
17. Examine Row M_5: Unvisited destinations M_1, M_2.
18. Times: $M_1 = \mathbf{1.1}, M_2 = 1.3$.
19. Minimum is M_1 with $c_{M_5,M_1} = \mathbf{1.1}$.
20. Tour so far: $S \to M_3 \to M_4 \to M_5 \to M_1$. Visited: S, M_3, M_4, M_5, M_1.
21. **At Machine M1**:
22. Only unvisited machine remaining is M_2.
23. Row M_1 travel time to M_2: $c_{M_1,M_2} = \mathbf{2.7}$.
24. Tour so far: $S \to M_3 \to M_4 \to M_5 \to M_1 \to M_2$. All machines visited.
25. **Return Leg to Storage Room S**:
26. Examine travel time from last visited node M_2 back to origin S.
27. Row M_2 time to S: $c_{M_2,S} = \mathbf{2.1}$.

Total Travel Time:

$$\textbf{Sequence } \mathbf{S_1}: S \to M_3 \to M_4 \to M_5 \to M_1 \to M_2 \to S$$

$$\mathbf{T_1} = 0.7 + 1.0 + 1.4 + 1.1 + 2.7 + 2.1 = \mathbf{9.0} \text{ minutes}$$

6.3 Question 4.2: Sequence S2 from Machine M1 ($T_2 = 7.9$ min)

Problem Statement:

> Assuming that the AGV is currently in machine center M_1 and starts its journey from that station, use the nearest-neighbor method again to generate a production sequence S_2 and calculate the corresponding total changeover time T_2.

Step-by-Step Traversal:

1. **At Machine M1**:
2. Examine Row M_1: Destinations S, M_2, M_3, M_4, M_5.
3. Times: $S = \mathbf{1.8}, M_2 = 2.7, M_3 = 2.3, M_4 = 2.5, M_5 = 2.0$.
4. Minimum is S with $c_{M_1,S} = \mathbf{1.8}$.
5. Tour so far: $M_1 \to S$. Visited: M_1, S.
6. **At Storage Room S**:
7. Examine Row S: Unvisited destinations M_2, M_3, M_4, M_5.
8. Times: $M_2 = 1.4, M_3 = \mathbf{0.7}, M_4 = 1.1, M_5 = 1.6$.
9. Minimum is M_3 with $c_{S,M_3} = \mathbf{0.7}$.
10. Tour so far: $M_1 \to S \to M_3$. Visited: M_1, S, M_3.
11. **At Machine M3**:
12. Examine Row M_3: Unvisited destinations M_2, M_4, M_5.
13. Times: $M_2 = 1.6, M_4 = \mathbf{1.0}, M_5 = 1.9$.
14. Minimum is M_4 with $c_{M_3,M_4} = \mathbf{1.0}$.
15. Tour so far: $M_1 \to S \to M_3 \to M_4$. Visited: M_1, S, M_3, M_4.
16. **At Machine M4**:
17. Examine Row M_4: Unvisited destinations M_2, M_5.
18. Times: $M_2 = 1.9, M_5 = \mathbf{1.4}$.
19. Minimum is M_5 with $c_{M_4,M_5} = \mathbf{1.4}$.
20. Tour so far: $M_1 \to S \to M_3 \to M_4 \to M_5$. Visited: M_1, S, M_3, M_4, M_5.
21. **At Machine M5**:
22. Only unvisited machine remaining is M_2.
23. Row M_5 travel time to M_2: $c_{M_5,M_2} = \mathbf{1.3}$.
24. Tour so far: $M_1 \to S \to M_3 \to M_4 \to M_5 \to M_2$. All machines visited.
25. **Return Leg to Machine M1**:
26. Travel from last visited node M_2 back to starting node M_1.
27. Row M_2 time to M_1: $c_{M_2,M_1} = \mathbf{1.7}$.

Total Travel Time:

$$\textbf{Sequence } \mathbf{S_2}: M_1 \to S \to M_3 \to M_4 \to M_5 \to M_2 \to M_1$$

$$\mathbf{T_2} = 1.8 + 0.7 + 1.0 + 1.4 + 1.3 + 1.7 = \mathbf{7.9} \text{ minutes}$$

6.4 Question 4.3: Diagnostic True / False / MTF Analysis

Problem Statement:

> Based on the results in 4.1 ($T_1 = 9.0$) and 4.2 ($T_2 = 7.9$), answer the following questions by placing **T (True)**, **F (False)**, or **MTF (May be True or False)**. You do not need to give any explanation.

Part	Statement	Answer	Rigorous Mathematical & Pedagogical Proof
a	S1 is optimal.	F	Sequence S2 visits the exact same 6 locations in 7.9 minutes. An optimal solution achieves the global minimum time: T* ≤ T for all feasible tours. Since T2 = 7.9 < 9.0 = T1, S1 is definitively sub-optimal.
b	S2 is optimal.	MTF	Nearest-Neighbor is a myopic greedy heuristic. Although S2 improved upon S1, it does not guarantee the global optimum. Without evaluating all (n-1)! = 120 candidate tours, one cannot assert whether S2 is globally optimal or not.
c	S1 is NOT optimal.	T	Statement (c) is the exact logical negation of Statement (a). Since "S1 is optimal" is False, "S1 is NOT optimal" is True.
d	S2 is NOT optimal.	MTF	Just as we cannot prove S2 is optimal without a full search, we cannot prove it is NOT optimal without finding a strictly superior tour. Thus, stating it is NOT optimal is also MTF.

6.5 Question 4.4: The Multi-Start Nearest Neighbor Controversy

Problem Statement 4.4:

> *"If we follow the same nearest neighbor method to generate 6 sequences S_1, S_2, S_3, S_4, S_5, and S_6 starting from S, M_1, M_2, M_3, M_4, and M_5, respectively, and calculate total travel time for each of T_1, T_2, T_3, T_4, T_5, and T_6, then the sequence corresponding to the smallest value of T_1, T_2, T_3, T_4, T_5, and T_6 will be the optimal sequence of all possible sequences to deliver the material to all machines and return to the storage room."* **Indicate if the above statement is T (True), F (False) or MTF (May be True or False)**.

The Teacher's Exam Key vs. Theoretical Operations Research:

- **The Teacher's Grading Key**: **T (True)**

- *Contextual Rationale*: On the examination paper, the professor marked **True** because in the applied heuristic decision protocol taught in class, running the heuristic from every candidate vertex and selecting the best performer is the designated procedure to determine the operational solution for the facility.
- **The Theoretical Operations Research Reality**: **F (False) / MTF**
- *Mathematical Rationale*: The TSP is NP-hard. Generating 6 heuristic sequences tests only **6 candidate tours** out of the $(6-1)! = \mathbf{120}$ total directed permutations.
- A greedy heuristic can easily miss the true global optimum from all starting points if the optimal tour requires accepting a slightly higher local step early in order to unlock massive savings later.

6.6 Exhaustive Algorithmic Benchmark (All 120 Tours Proven)

To provide complete pedagogical closure and prove the limits of greedy heuristics, an exhaustive evaluation of all $(6-1)! = 120$ possible tours was computed:

$$\begin{array}{l|c|c} \text{Starting Node (Heuristic)} & \text{Heuristic Tour Path} & \text{Total Time} \ \hline \text{Start from } S\ (S_1) & S \to M_3 \to M_4 \to M_5 \to M_1 \to M_2 \to S & 9.0\text{ min} \ \text{Start from } M_1\ (S_2) & M_1 \to S \to M_3 \to M_4 \to M_5 \to M_2 \to M_1 & 7.9\text{ min} \ \text{Start from } M_2 & M_2 \to M_3 \to M_4 \to S \to M_1 \to M_5 \to M_2 & 7.0\text{ min} \ \text{Start from } M_3 & M_3 \to M_4 \to S \to M_1 \to M_5 \to M_2 \to M_3 & 7.0\text{ min} \ \text{Start from } M_4 & M_4 \to S \to M_3 \to M_1 \to M_5 \to M_2 \to M_4 & 8.3\text{ min} \ \text{Start from } M_5 & M_5 \to M_1 \to S \to M_3 \to M_4 \to M_2 \to M_5 & 7.8\text{ min} \ \hline \mathbf{Global\ Absolute\ Minimum} & \mathbf{S \to M_1 \to M_5 \to M_2 \to M_3 \to M_4 \to S} & \mathbf{7.0\text{ min}} \end{array}$$

Key Observations:

1. When starting from M_2 or M_3, the heuristic happens to find a closed loop of length **7.0 minutes**, which is indeed the global optimum!
2. However, when starting from the actual designated storage depot S, the nearest neighbor heuristic produced **9.0 minutes** — a staggering **28.6% sub-optimal penalty**!

---

7. The Industrial Engineer's Exam Diagnostic & Failure-Mode Prevention

STUDENT FAILURE MODE POST-MORTEM		
Error Trap	Erroneous Student Action	Correct Engineering Procedure
1. Column Navigation	Reading down Column M3 (1.6, 2.3) instead of Row M3 (1.3, 1.0, 1.9)	Always read across ROW i of current vertex to destination Columns j.
2. Forgetting Return	Summing legs between machines only and forgetting final leg to start	A closed TSP tour must always add the final leg back to origin vertex!
3. Re-visiting Nodes	Picking a low cost cell to a node that was already visited	Maintain a strict "Visited" set; candidates are ONLY unvisited nodes!
4. Premature Optimum	Declaring a greedy heuristic tour "definitely optimal"	Heuristics give feasible bounds; optimality without proof is MTF.
5. Symmetric Division	Dividing asymmetric tours by 2 (writing (n-1)! / 2)	For asymmetric matrices ($d_{ij} \neq d_{ji}$) directed tours are strictly (n-1)!

Concordia University · Gina Cody School of Engineering · Department of Mechanical, Industrial & Aerospace Engineering